

Exemplo - Módulo 3: Arquiteturas Multi-Agent (resolvido com TrialForge)

Disciplina: Arquitetura de Sistemas com IA · Prof. Ahirton Lopes, Ph.D.

Missão Prática #03 - solução de referência

Sistema: os três agentes especialistas do TrialForge (Protocolo, ICF, CSR) coordenados por um Supervisor, gerando os documentos de um estudo clínico completo.

Passo 1 - Padrões mapeados

- Protocolo → (ICF, CSR): SEQUENTIAL - ICF e CSR dependem dos critérios que o Protocolo define
- ICF || CSR: PARALLEL - nenhum depende do resultado do outro
- (Protocolo, ICF, CSR) → Verificação: SUPERVISOR - decisão central de comparar os três e resolver divergência
- Hierarchical: não se aplica a este sistema específico - os três especialistas reportam direto a um único Supervisor, sem sub-times por domínio. Seria o padrão certo se Protocolo, ICF e CSR tivessem, cada um, sua própria equipe de especialistas abaixo (a estrutura Orquestrador Raiz → Supervisores de domínio → especialistas vista no Módulo 3.3).
- Group Chat: não se aplica ao fluxo padrão implementado aqui, onde a consistência é resolvida centralmente pelo Supervisor. Aplicaria se ICF e CSR precisassem debater diretamente uma classificação de evento adverso, como no cenário do Módulo 3.3: nesse caso, os dois têm autoridade igual para discordar, e o Supervisor cede o papel de moderador de conversa em vez de árbitro único.
- Agente ICF, no meio da tarefa, detecta risco bioético fora de escopo → Agente Bioética: HANDOFF - transferência completa de controle, com contexto acumulado

Passo 2 - Falha e compensação por agente

Protocolo: timeout 30s, retry 3x. Se esgotar (CAP): prioriza Consistência - espera, porque os outros dois agentes dependem diretamente dele; seguir sem resultado aqui quebraria a cadeia inteira.

ICF: timeout 45s, retry 2x. Se esgotar: prioriza Disponibilidade - o Supervisor segue com o CSR e sinaliza o ICF como pendente, em vez de travar o sistema inteiro esperando.

CSR: timeout 60s, retry 2x. Mesma lógica de Disponibilidade do ICF.

Idempotência: todas as três operações de geração de documento são idempotentes - gerar o mesmo documento duas vezes com o mesmo input produz o mesmo resultado, sem efeito colateral duplicado (não envia notificação duplicada, não grava duas entradas no histórico).

Compensação (Saga): se o CSR (Passo 3) descobre uma inconsistência que invalida um critério do Protocolo (Passo 1), a compensação NÃO apaga o Protocolo v1 - ela versiona para v2 e aciona um Handoff para o Agente ICF regenerar apenas a seção afetada, preservando o que não mudou.

Passo 3 - Código JavaScript de comunicação (EventEmitter)

Contrato dos eventos (declarado antes do código, como pede o Passo 3 da Atividade):

protocolo:pronto - emitido por Agente Protocolo; carrega { versao, criterios }; escutado por Agente ICF e Agente CSR.

supervisor:verificar - emitido pelo handler de protocolo:pronto, após ICF e CSR concluírem em paralelo; carrega { icf, csr, versaoProtocolo }; escutado por Agente Supervisor.

protocolo:revisar - emitido por Agente Supervisor (compensação), só quando verificarConsistencia encontra divergência; carrega { motivo }; escutado por Agente Protocolo.

```
const filaDeMensagens = new EventEmitter();
```

```
async function agenteProtocolo(estudo) {  
  const criterios = await gerarCriterios(estudo);  
  filaDeMensagens.emit('protocolo:pronto', { versao: 1, criterios });  
}
```

```
filaDeMensagens.on('protocolo:pronto', async (msg) => {  
  const [icf, csr] = await Promise.all([  
    agenteICF(msg.criterios),  
    agenteCSR(msg.criterios)  
  ]);  
  filaDeMensagens.emit('supervisor:verificar', { icf, csr, versaoProtocolo: msg.versao });  
});
```

```
filaDeMensagens.on('supervisor:verificar', async (msg) => {  
  const divergencia = await verificarConsistencia(msg.icf, msg.csr);  
  if (divergencia) {
```

```
        filaDeMensagens.emit('protocolo:revisar', { motivo: divergencia });  
    }  
});
```

Nota de curadoria

Repare que Protocolo e ICF/CSR têm timeouts diferentes (30s vs 45-60s) porque as tarefas são genuinamente diferentes em custo computacional - não é um valor genérico copiado entre agentes. E a escolha de Consistência para o Protocolo, mas Disponibilidade para ICF/CSR, reflete a posição de cada um na cadeia de dependência: falhar cedo é mais caro do que falhar tarde.

Reflexão final - um sinal de mudança

Sinal de mudança: se a compensação do Passo 3 (o CSR invalidando um critério que o Protocolo já tinha fixado) disparar com frequência alta, é sinal de que a divisão de responsabilidade entre Protocolo e CSR está errada: o critério deveria ser validado mais cedo, antes do CSR ser gerado, não depois. Compensação frequente não é falha do padrão Saga, é sintoma de que o limite entre os agentes foi desenhado no lugar errado.

Ahirton Lopes · AI Architecture Toolkit - UNIPDS: Arquitetura de Sistemas com IA

Prof. Ahirton Lopes, Ph.D. - GDE AI, Microsoft MVP, Senior Manager